Aim :Implement Breadth-First Search (BFS) on a Graph Using Python

Program:

```
print("Hello I am Treasure Hunt Robot!")
print("Let Us search the Treasure together")

Map_rooms = {

   "A": ["B", "C"],
   "B": ["D"],
   "C": ["D"],
   "D": ["G"], "G":
   []
}


map_rooms = {
   "A": ["B", "C"],
   "B": ["D"],
   "C": ["D"],
   "D": ["G"], "G":
   []
}

print("        A          ")
print("       / \\         ")
print("      B C          ")
print("      \\ /          ")
print("       D           ")
print("       |          ")
print("       G           ")

def bfs(start, goal):
to_do = [start]
visited = []
order = []
while to_do:
      room = to_do.pop(0)
      if room in visited:
         continue
      visited.append(room)
      order.append(room)
      if room == goal:
      return order
      for nxt in map_rooms[room]:
         to_do.append(nxt)
   return order

order_bfs = bfs("A", "G")
print("BFS checked the rooms in this order:", order_bfs)
print("Number of rooms BFS checked:", len(order_bfs))
```

```
def dfs(start, goal):
    to_do = [start]
    visited = []
    order = []
    while to_do:
        room = to_do.pop()
        if room in visited:
            continue
        visited.append(room)
        order.append(room)
        if room == goal:
        return order
        fornxtinmap_rooms[room]:
            to_do.append(nxt)
    return order

order_dfs = dfs("A", "G")
print("DFScheckedtherooms in this order:", order_dfs)
print("NumberofroomsDFS checked:", len(order_dfs))


door_cost = {
    ("A", "B"): 1,
    ("A", "C"): 1,
    ("B", "D"): 1,
    ("C", "D"): 5,
    ("D", "G"): 1,
}

def path_cost(path):
    total = 0
    foriinrange(len(path)-1):
        door=(path[i],path[i+ 1])
        total=total+door_cost[door]
    return total

path1=["A","B","D","G"]
path2=["A","C","D","G"]

print("Path1A->B->D-> G costs:", path_cost(path1))
print("Path2A->C->D-> G costs:", path_cost(path2))

cost1 = path_cost(path1)
cost2 = path_cost(path2)

ifcost1 < cost2:
    best_path = path1
    best_cost = cost1
else:
    best_path = path2
    best_cost = cost2
```

```
print("Path 1 cost:", cost1)
print("Path 2 cost:", cost2)
print("The cheapest path is", best_path, "with cost", best_cost, )

bfs_order = dfs("A", "G")
dfs_order = dfs("A", "G")
```

Output:

```
Hello I am Treasure Hunt Robot!
Let Us search the Treasure together

          A
         / \
        B   C
        \   /
          D
          |
          G
BFS checked the rooms in this order: ['A', 'B', 'C', 'D', 'G']
Number of rooms BFS checked: 5
DFS checked the rooms in this order: ['A', 'C', 'D', 'G']
Number of rooms DFS checked: 4

Path 1 A -> B -> D -> G costs: 3
Path 2 A -> C -> D -> G costs: 7
Path 1 cost: 3
Path 2 cost: 7
The cheapest path is ['A', 'B', 'D', 'G'] with cost 3
```